

Demo Runbook 2

Unit Testing • ML Code Validation • Experiment Tracking • Model Registry

Copy-paste-ready commands, expected outputs, and talking points for running each lab live as a demo.

How to use this	Run the Setup block once before the audience arrives. During the demo, execute each numbered Step live, pausing on the 'Expected output' box so the audience sees the result before you move on.
Total demo time	Approx. 30-40 minutes for all 4 labs

LAB 1

Unit Testing

Showing pytest catch a real bug live, not just pass green

Demo Goal: Prove that a test suite actually protects against regressions by breaking the code live and watching the test fail.

Setup (run once before the demo)

```
pip install pytest pandas
mkdir unit_testing_demo/src unit_testing_demo/tests -p && cd unit_testing_demo
```

STEP 1

Write the function under test

```
cat > src/preprocessing.py << 'EOF'
import pandas as pd

def normalize_column(df: pd.DataFrame, column: str) -> pd.DataFrame:
    df = df.copy()
    col = df[column]
    if col.max() == col.min():
        raise ValueError('Cannot normalize a constant column')
    df[column] = (col - col.min()) / (col.max() - col.min())
    return df
EOF
```

STEP 2

Write the test suite

```
cat > tests/test_preprocessing.py << 'EOF'
import pandas as pd, pytest
from src.preprocessing import normalize_column

def test_normalize_range():
    df = pd.DataFrame({'x': [10, 20, 30, 40]})
    result = normalize_column(df, 'x')
    assert result['x'].min() == 0.0
    assert result['x'].max() == 1.0

def test_constant_column_raises():
    df = pd.DataFrame({'x': [5, 5, 5]})
    with pytest.raises(ValueError):
        normalize_column(df, 'x')
EOF
touch src/__init__.py
```

STEP 3

Run the suite — all green

```
pytest -v
```

Expected output / what you'll see:

```
tests/test_preprocessing.py::test_normalize_range PASSED
tests/test_preprocessing.py::test_constant_column_raises PASSED
===== 2 passed in 0.05s =====
```

STEP 4

Break the code live in front of the audience

Remove the constant-column guard from src/preprocessing.py (delete the if/raise block) — simulating a careless future code change.

```
pytest -v
```

Expected output / what you'll see:

```
tests/test_preprocessing.py::test_constant_column_raises FAILED
E Failed: DID NOT RAISE
===== 1 failed, 1 passed =====
```

Say to audience: *"This is the entire value proposition of unit testing in one moment: a real regression, caught instantly, before it ever reaches a pull request review or production. Now watch me put the guard back and re-run."*

STEP 5

Fix it back and confirm green again

```
# restore the if/raise block, then:
pytest -v
```

Expected output / what you'll see:

```
===== 2 passed in 0.05s =====
```

Demo Wrap-up

Keep the broken version in your terminal history (up-arrow) so you can re-break/re-fix quickly if the audience wants to see it twice.

LAB 2

ML Code Validation

Catching bad data before it reaches training, live

Demo Goal: Feed intentionally bad data into a validation function and show it get rejected with a clear error, instead of silently corrupting a downstream model.

Setup (run once before the demo)

```
pip install pandas pytest
mkdir -p code_validation_demo/src && cd code_validation_demo
```

STEP 1

Write the validation checks

```
cat > src/data_validation.py << 'EOF'
import pandas as pd

def validate_ranges(df: pd.DataFrame) -> list:
    errors = []
    if (df['age'] < 0).any() or (df['age'] > 120).any():
        errors.append('age out of valid range [0, 120]')
    if (df['income'] < 0).any():
        errors.append('income cannot be negative')
    return errors

def validate_missing(df: pd.DataFrame, max_missing_frac=0.05) -> list:
    errors = []
    for col in df.columns:
        frac = df[col].isnull().mean()
        if frac > max_missing_frac:
            errors.append(f'{col} has {frac:.0%} missing (limit {max_missing_frac:.0%})')
    return errors
EOF
```

STEP 2

Run validation on clean data first

```
cat > run_check.py << 'EOF'
import pandas as pd
from src.data_validation import validate_ranges, validate_missing

df = pd.DataFrame({'age': [25, 40, 55], 'income': [50000.0, 62000.0, 71000.0]})
errors = validate_ranges(df) + validate_missing(df)
print('Errors found:', errors if errors else 'NONE – data is clean')
EOF
touch src/__init__.py
python run_check.py
```

Expected output / what you'll see:

Errors found: NONE – data is clean

STEP 3

Now feed it deliberately bad data

```
cat > run_check_bad.py << 'EOF'
import pandas as pd
from src.data_validation import validate_ranges, validate_missing
```

```
bad_df = pd.DataFrame({'age': [-5, 200, 40], 'income': [-1000.0, 50000.0, None]})
errors = validate_ranges(bad_df) + validate_missing(bad_df)
print('Errors found:', errors)
if errors:
    raise SystemExit('Blocking pipeline: data failed validation')
EOF
python run_check_bad.py
```

Expected output / what you'll see:

```
Errors found: ['age out of valid range [0, 120]', 'income cannot be negative']
SystemExit: Blocking pipeline: data failed validation
```

Say to audience: *"This is what stops a corrupted dataset from silently training a broken model — the pipeline stops here, loudly, instead of producing a model that quietly performs badly in production three weeks from now."*

Demo Wrap-up

If time allows, mention this same pattern is what tools like Great Expectations or Pandera automate at scale — this demo shows the underlying principle in plain Python.

LAB 3

Experiment Tracking

Running several training variants and comparing them live in the MLflow UI

Demo Goal: Run 3-4 quick training variations and show the audience how to instantly compare them side by side, instead of scrolling through terminal output.

Setup (run once before the demo)

```
pip install mlflow scikit-learn
mkdir experiment_tracking_demo && cd experiment_tracking_demo
```

STEP 1

Write a parameterized training script

```
cat > train.py << 'EOF'
import sys, mlflow
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

n_estimators = int(sys.argv[1]) if len(sys.argv) > 1 else 100
max_depth = int(sys.argv[2]) if len(sys.argv) > 2 else 5

mlflow.set_experiment('demo-tracking')
X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

with mlflow.start_run():
    mlflow.log_param('n_estimators', n_estimators)
    mlflow.log_param('max_depth', max_depth)
    model = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth).fit(X_train, y_train)
    preds = model.predict(X_test)
    mlflow.log_metric('accuracy', accuracy_score(y_test, preds))
    mlflow.log_metric('f1_macro', f1_score(y_test, preds, average='macro'))
    mlflow.sklearn.log_model(model, 'model')
    print(f'n_estimators={n_estimators}, max_depth={max_depth} logged.')
EOF
```

STEP 2

Run four quick variants live, back to back

```
python train.py 50 3
python train.py 100 5
python train.py 200 8
python train.py 300 None 2>/dev/null || python train.py 300 20
```

Expected output / what you'll see:

```
n_estimators=50, max_depth=3 logged.
n_estimators=100, max_depth=5 logged.
n_estimators=200, max_depth=8 logged.
n_estimators=300, max_depth=20 logged.
```

STEP 3

Open the MLflow UI and sort by accuracy

```
mlflow ui # open http://127.0.0.1:5000, select 'demo-tracking', sort by accuracy column
```

Expected output / what you'll see:

All 4 runs listed with their exact parameters and metrics side by side, sortable by clicking any metric column header.

Say to audience: "Notice I didn't have to remember which terminal output had which numbers — every run is permanently recorded with the exact settings that produced it. This is what makes it possible to answer 'what was our best run and how do we reproduce it' months later."

STEP 4

Select two runs and use the Compare view

Check the boxes next to the best and worst run, click 'Compare'.

Expected output / what you'll see:

A side-by-side parameter/metric diff view, and a parallel-coordinates plot showing how `n_estimators`/`max_depth` relate to accuracy.

Say to audience: "This visual is often the moment that lands best with a non-technical audience — it turns four terminal outputs into one clear comparison."

LAB 4

Model Registry

Registering a model and promoting it through Staging to Production live

Demo Goal: Show the audience how a specific tracked run becomes a governed, versioned, stage-managed model that downstream code can reference by name instead of by run ID.

Setup (run once before the demo)

```
pip install mlflow scikit-learn
(Run Lab 3 first, or re-run train.py a couple of times so runs exist)
```

STEP 1

Register the best run as a named model

```
cat > register_model.py << 'EOF'
import mlflow
from mlflow.tracking import MlflowClient

client = MlflowClient()
exp = client.get_experiment_by_name('demo-tracking')
best_run = client.search_runs(exp.experiment_id, order_by=['metrics.accuracy DESC'], max_results=1)[0]
print('Best run:', best_run.info.run_id, 'accuracy:', best_run.data.metrics['accuracy'])

result = mlflow.register_model(f'runs://{best_run.info.run_id}/model', 'wine-demo-model')
print('Registered as version:', result.version)
EOF
python register_model.py
```

Expected output / what you'll see:

```
Best run: accuracy: 0.97+
Registered as version: 1
```

Say to audience: "This model now has a permanent name — 'wine-demo-model' — decoupled from the specific run ID that produced it."

STEP 2

Open the Models tab in the MLflow UI

```
mlflow ui # click 'Models' tab -> wine-demo-model
```

Expected output / what you'll see:

```
wine-demo-model, Version 1, currently in stage 'None', linked back to its source run.
```

STEP 3

Promote the model to Staging, then Production

```
cat > promote.py << 'EOF'
from mlflow.tracking import MlflowClient
client = MlflowClient()

client.transition_model_version_stage(name='wine-demo-model', version=1, stage='Staging')
print('Moved to Staging')

client.transition_model_version_stage(name='wine-demo-model', version=1, stage='Production',
                                     archive_existing_versions=True)
print('Promoted to Production')
```

```
EOF
python promote.py
```

Expected output / what you'll see:

```
Moved to Staging
Promoted to Production
```

Say to audience: "In a real pipeline, that Staging step would be gated behind automated validation, and Production behind a human or automated sign-off — we're compressing that for the demo."

STEP 4

Load the model by stage name, not by run ID

```
cat > load_production.py << 'EOF'
import mlflow.pyfunc
from sklearn.datasets import load_wine

model = mlflow.pyfunc.load_model('models:/wine-demo-model/Production')
X, _ = load_wine(return_X_y=True)
print('Prediction:', model.predict(X[:1]))
EOF
python load_production.py
```

Expected output / what you'll see:

```
Prediction: [0]
```

Say to audience: "This is the payoff: this code will keep working even if we register a completely new model version tomorrow and promote it — because it points at the **stage**, not a specific version. That's what lets a model be swapped in production without touching downstream code."

Demo Wrap-up

This closes the loop from Lab 3: an experiment run becomes a governed, versioned, stage-managed artifact that other services can safely depend on.